

Lab 09 - Enterprise Application Integration

Integrating Independent Applications using REST APIs

Student Name:	Student ID:
Group:	Date:

1. Purpose of the Lab

This lab gives you a small hands-on example of Enterprise Application Integration (EAI). You will create two independent applications and integrate them using a REST API. The activity demonstrates application-level integration through remote invocation, with JSON used as the shared data format.

2. Learning Outcomes

- Identify two independent applications that require integration.
- Create and expose a simple REST API.
- Consume a REST API from another application.
- Exchange data between applications using JSON.
- Test an integration using an API testing tool.
- Explain basic EAI concerns such as coupling, data format, timeliness, and availability.

3. Scenario

A small online shopping company has two independent systems: an Order Management System and an Inventory Management System. At present, when an order is placed, an employee manually checks whether the product is available. Your task is to integrate these systems so that the order application checks product availability automatically.

Order Management System	Inventory Management System
Receives customer orders and decides whether an order can be accepted.	Stores product details and current stock availability.

Example Order Data

```
{
  "orderId": 101,
  "customerName": "Nimal",
  "product": "Laptop",
  "quantity": 1
}
```

Example Inventory Data

```
{
  "productId": 1,
  "productName": "Laptop",
  "quantity": 15
}
```

4. Lab Architecture

Customer -> Order Management System -> REST / JSON -> Inventory Management System

The Order Management System acts as the API consumer. The Inventory Management System exposes product availability through a REST endpoint.

5. Task 1 - Create the Inventory API

Create a small REST API for the Inventory Management System. You may use any suitable backend technology, for example:

- Spring Boot
- ASP.NET Core
- Node.js with Express
- Python Flask or FastAPI

For this lab, a database is not required. You may keep a small list/array of products in memory.

Required Endpoint

```
GET /api/inventory/{productName}
```

Example request:

```
GET /api/inventory/Laptop
```

Example response:

```
{
  "productName": "Laptop",
  "quantity": 15,
  "available": true
}
```

6. Task 2 - Test the Inventory API

Use Postman, Swagger, Thunder Client, or a similar tool to test the endpoint. Test at least the following requests:

- /api/inventory/Laptop
- /api/inventory/Phone
- /api/inventory/UnknownProduct

Evidence required: Capture at least one screenshot showing a successful API response.

7. Task 3 - Integrate the Order Application

Create an order endpoint in the Order Management System:

```
POST /api/orders
```

Example request body:

```
{
  "customerName": "Nimal",
  "product": "Laptop",
  "quantity": 2
}
```

Before accepting the order, the Order Management System must call the Inventory API to check whether the requested quantity is available.

1. Customer sends an order
2. Order Management System receives the order
3. Order system calls the Inventory REST API
4. Inventory quantity is checked
5. Order is accepted or rejected

If sufficient stock is available, return:

```
{
  "message": "Order placed successfully"
}
```

If sufficient stock is not available, return:

```
{
  "message": "Insufficient stock"
}
```

8. Task 4 - Test the Integration

Run the following test cases and record the observed result.

Test	Quantity Requested	Expected Result	Observed Result
Laptop	2	Order accepted	
Laptop	100	Insufficient stock	
UnknownProduct	1	Product not found	

Evidence required: Capture screenshots of at least one successful order and one rejected order.

9. Task 5 - Integration Analysis

Answer each question briefly (1-3 sentences).

1. What EAI approach is used in this lab?

Answer: _____

2. What data format is exchanged between the two applications?

Answer: _____

3. Is this integration synchronous or asynchronous? Explain briefly.

Answer: _____

4. What happens if the Inventory Management System is unavailable?

Answer: _____

5. State one advantage of this integration approach.

Answer: _____

6. State one disadvantage of this integration approach.

Answer: _____

7. How does this design create coupling between the two applications?

Answer: _____

10. Deliverables

- Source code for the Order Management System and Inventory Management System.
- Screenshot of the Inventory API response.
- Screenshot of a successful order.
- Screenshot of a rejected order.
- A simple integration architecture diagram.
- Completed answers for the Integration Analysis questions.

11. Optional Challenge

Redesign the same integration using Messaging instead of direct REST invocation. You do not need to implement it. Draw a small architecture diagram and explain how the communication would change if a message queue were introduced.

Order Service -> Message Queue -> Inventory Service